



DSP404 · Unidad I (cierre)

Métodos y Funciones

S e m a n a 4

Métodos, DRY y Descomposición

DRY

Parámetros

Retorno

Sobrecarga

Defecto

Temas de hoy:

- El problema que resuelven los métodos (DRY)
- Anatomía: tipo retorno, nombre, parámetros
- void vs métodos con retorno
- Sobrecarga de métodos
- Parámetros con valor por defecto
- Descomposición de problemas

¿Qué lograrás al terminar esta clase?



Explicas el principio DRY y por qué existe

Don't Repeat Yourself — el problema real antes de la solución.

Defines métodos con y sin valor de retorno

void para acciones, tipo (int, double...) para cálculos.

Usas parámetros para pasar datos a un método

Diferencia entre parámetro (definición) y argumento (llamada).

Sobrecargas un método con distintas firmas

Mismo nombre, distintos parámetros — C# elige el correcto.

Descompones un programa en métodos cohesivos

Cada método hace UNA cosa bien. El código se vuelve legible.

Bloque 1

El Problema — DRY

Don't Repeat Yourself: no repitas código

¿Por qué existen los métodos? — el problema



DRY = Don't Repeat Yourself. Si escribes el mismo código dos veces, algo está mal. Los métodos son la solución.



Sin métodos — código repetido

CS

```
// Calcular IVA para producto 1
double precio1 = 150.0;
double iva1 = precio1 * 0.13;
double total1 = precio1 + iva1;
Console.WriteLine($"Total:
{total1:C}");

// Calcular IVA para producto 2
double precio2 = 89.99;
double iva2 = precio2 * 0.13; //
repetido
double total2 = precio2 + iva2; //
repetido
```



Con métodos — DRY y mantenible

CS

```
// El método se define UNA vez
static double TotalConIVA(double
precio)
{
    return precio * 1.13;
}

// Se usa cuantas veces se necesite
double total1 = TotalConIVA(150.0);
double total2 = TotalConIVA(89.99);
double total3 = TotalConIVA(299.0);

Console.WriteLine($"Total:
{total1:C}");
```



Regla de oro: si copias y pegas código, ahí va un método. Si encuentras el mismo bloque 2+ veces, refactoriza.

Bloque 2

Anatomía de un Método

Las partes que todo método tiene

Las partes de un método — antes de escribir código

```
static double CalcularPromedio ( double[] notas ,  
int cantidad )
```

Modificador

static = se llama sin
crear objeto

Tipo retorno

double = devuelve un
decimal

Nombre

Descriptivo, verbo en
infinitivo

Parámetros

Datos que recibe para trabajar

CS

```
static double CalcularPromedio(double[] notas, int cantidad)  
{  
    // ← Cuerpo del método: la lógica  
    double suma = 0;  
    foreach (double n in notas) suma += n;  
    return suma / cantidad; // ← return devuelve el resultado  
}
```



LLAMADA (cómo se usa)

Parámetro = la variable en la definición del método. Argumento = el valor que pasas al llamarlo. Son palabras distintas

Parámetros — cómo los datos entran al método

CS

```
// — SIN parámetros

static void MostrarMenu()
{
    Console.WriteLine("1. Sumar  2. Restar  3. Salir");
}

// — CON un parámetro

static double CirculoArea(double radio)
{
    return Math.PI * radio * radio;
}

// — CON múltiples parámetros

static double TotalFactura(double precio, int cantidad, double descuento)
{
    double subtotal = precio * cantidad;
```

Reglas de parámetros



Los tipos van antes del nombre

(double radio), no solo (radio)



Se pasan por valor

C# copia el valor — el original no cambia



El orden importa

(precio, cantidad) ≠ (cantidad, precio)



Deben coincidir en cantidad

3 params en definición = 3 args en llamada



Nombrarlos con claridad

precioUnitario, no p1

Bloque 3

Sobrecarga y Parámetros por Defecto

Mismo nombre, distinto comportamiento

Sobrecarga de métodos — ¿qué es y para qué sirve?



La sobrecarga permite tener VARIOS métodos con el MISMO nombre pero DISTINTOS parámetros. C# elige automáticamente cuál usar según lo que le pases.



Analogía del mundo real: la función 'Imprimir' en Word funciona igual si imprimes 1 página, si seleccionas un rango, o si imprimes todo el documento. Mismo nombre, distintos parámetros.

```
static double Area(double lado)
```

→ llamada: Area(5.0) → cuadrado

25.0

```
static double Area(double base, double altura)
```

→ llamada: Area(4.0, 6.0) → rectángulo

24.0

```
static double Area(double radio, bool esCirculo)
```

→ llamada: Area(3.0, true) → círculo

28.27

Sobrecarga y parámetros por defecto — código completo

CS

```
// — SOBRECARGA: mismo nombre, distintos parámetros —————
static double Area(double lado)                                // cuadrado
    => lado * lado;

static double Area(double b, double h)                        // rectángulo
    => b * h;

static double Area(double radio, bool esCirculo)             // círculo
    => esCirculo ? Math.PI * radio * radio : radio * radio;

// C# elige según los argumentos:
Console.WriteLine(Area(5.0));                                // 25.0
Console.WriteLine(Area(4.0, 6.0));                            // 24.0
Console.WriteLine(Area(3.0, true));                           // 28.27

// — PARÁMETROS POR DEFECTO —————
static double Potencia(double base2, int exponente = 2)
    => Math.Pow(base2, exponente);

Console.WriteLine(Potencia(4));                                // 16   (exponente usa 2)
Console.WriteLine(Potencia(3, 3));                            // 27   (exponente = 3)
```



Los parámetros por defecto siempre van AL FINAL de la lista de parámetros. `static void M(int a = 1, int b)` es inválido: `static`

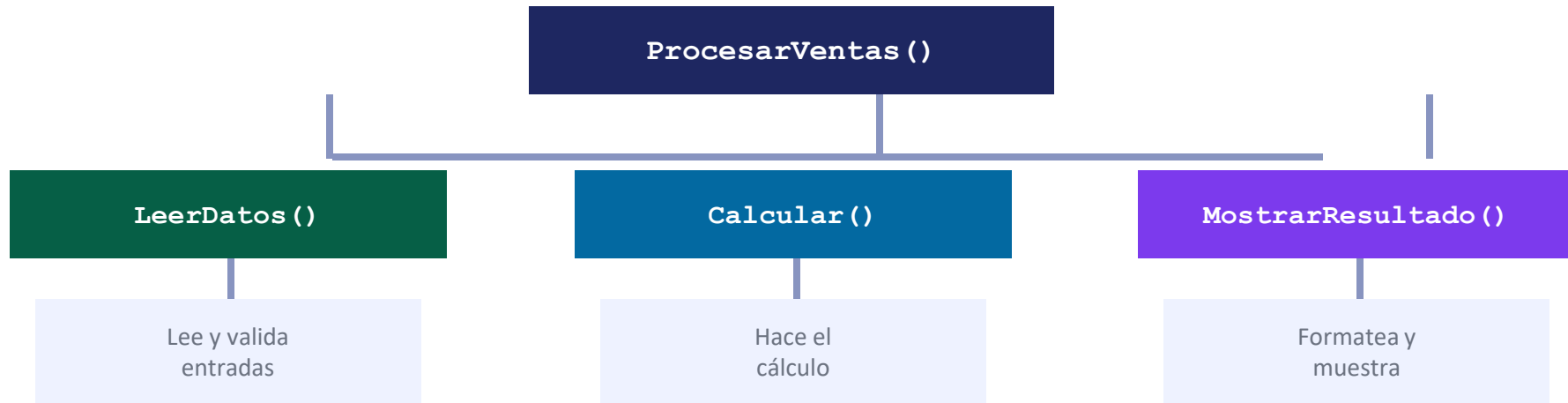
Bloque 4

Descomposición de Problemas

Dividir para vencer — el arte de diseñar métodos

Descomposición de problemas — el concepto

💡 **Descomponer = dividir un problema grande en subproblemas pequeños, cada uno resuelto por un método. Un método bien diseñado hace UNA sola cosa.**



Reglas para diseñar buenos métodos:

1 Una responsabilidad: Si el nombre necesita 'y' (calcular Y mostrar), son dos métodos

✂ Cortos (5-20 líneas): Si el método no cabe en pantalla, probablemente hace demasiado

📝 Nombre descriptivo: `CalcularIVA()` dice qué hace. `Metodo1()` no dice nada

Funciones predefinidas — Math, String y Console

CS

```
// — Math — operaciones matemáticas —————  
Console.WriteLine(Math.Abs(-15));           // 15      (valor absoluto)  
Console.WriteLine(Math.Round(3.7));         // 4        (redondeo)  
Console.WriteLine(Math.Floor(3.9));         // 3        (hacia abajo)  
Console.WriteLine(Math.Ceiling(3.1));       // 4        (hacia arriba)  
Console.WriteLine(Math.Sqrt(16));          // 4.0      (raíz cuadrada)  
Console.WriteLine(Math.Pow(2, 8));          // 256.0    (potencia)  
Console.WriteLine(Math.Max(18, 42));        // 42  
Console.WriteLine(Math.Min(18, 42));        // 18  
Console.WriteLine(Math.PI);                // 3.14159265...  
  
// — String — métodos sobre texto —————  
string s2 = "  Hola Mundo  ";  
Console.WriteLine(s2.ToUpper());           // "  HOLA MUNDO  "  
Console.WriteLine(s2.ToLower());           // "  hola mundo  "  
Console.WriteLine(s2.Trim());              // "Hola Mundo"  
Console.WriteLine(s2.Length);              // 14  
Console.WriteLine(s2.Contains("Mundo"));   // True  
Console.WriteLine(s2.Replace("Mundo", "C#")); // "  Hola C#  "  
Console.WriteLine(s2.Substring(7, 5));     // "Mundo"
```



Estos son métodos que ya vienen con .NET. No los tienes que memorizar todos — aprende a buscar en la documentación.



Demo en vivo — Calculadora Refactorizada

Tomamos la calculadora de la Semana 2 y la rediseñamos con métodos
El docente muestra la transformación paso a paso

Ejercicio explicado — Calculadora Refactorizada con Métodos

CS

```
// == CALCULADORA SEMANA 2 → REFACTORIZADA CON MÉTODOS ==  
  
// — Métodos de cálculo —  
static double Sumar(double a, double b) => a + b;  
static double Restar(double a, double b) => a - b;  
static double Multiplicar(double a, double b) => a * b;  
static double Dividir(double a, double b)  
    => b != 0 ? a / b : double.NaN;  
  
// — Métodos de UI —  
static void MostrarMenu()  
{  
    Console.WriteLine("  
=====");  
    Console.WriteLine("|| CALCULADORA ||");  
    Console.WriteLine("||-----||");  
    Console.WriteLine("|| 1.Sumar      2.Restar||");  
    Console.WriteLine("|| 3.Mult.      4.Dividir  0.Salir||");  
    Console.WriteLine("||-----||");  
}  
  
static bool LeerOpcion(out int opcion)
```




¡Tu turno! — Biblioteca de Funciones Matemáticas

30 minutos | Individual o en parejas
Lee el enunciado en el siguiente slide



EJERCICIO — Biblioteca de Funciones Matemáticas



Parte 1 — Métodos con retorno

- `static double AreaCirculo(double radio) →  $\pi \times r^2$`
- `static double AreaRectangulo(double base, double altura) →  $\text{base} \times \text{altura}$`
- `static double AreaTriangulo(double base, double altura) →  $(\text{base} \times \text{altura}) / 2$`
- `static bool EsPrimo(int n) → true si n es primo, false si no`
- `static int Factorial(int n) →  $n!$  usando un ciclo for`

Parte 2 — Métodos void (con sobrecarga)

- `static void MostrarResultado(string nombre, double valor) → imprime con 2 decimales`
- `static void MostrarResultado(string nombre, bool valor) → imprime true/false en texto`
- Llama ambas sobrecargas en el programa principal

💡 Pistas: para `EsPrimo`, prueba divisores del 2 hasta $\sqrt{n}$. Para `Factorial`, multiplica acumulando en un bucle for.



Práctica en CodeQuest C#

Abre la plataforma y completa la Lección 4: Métodos
Esta es la última lección de la Unidad I

[Pega aquí el enlace a la plataforma]

Errores comunes — Semana 4

Los que TODOS cometemos al aprender métodos

❌ Olvidar el return en un método con tipo de retorno

```
static int Doble(int x)
{
    int resultado = x * 2;
    // ❌ falta return
}
```

```
static int Doble(int x)
{
    int resultado = x * 2;
    return resultado; // ✅
```

🚫 CS0161: 'Doble': not all code paths return a value

❌ Llamar el método pero no usar el resultado

```
static double Promedio(double[] n) { ... }
// ...
Promedio(notas); // ❌ se calcula y se pierde
```

```
double prom = Promedio(notas); // ✅ guardarlo
Console.WriteLine(prom);       // ✅ o usarlo directo
```

🚫 No hay error de compilación — pero el resultado se descarta silenciosamente

❌ Confundir parámetros con variables globales

```
double precio = 50.0; // variable en Program
static double ConIVA()
    => precio * 1.13; // ❌ depende de variable externa
```

```
static double ConIVA(double precio)
    => precio * 1.13; // ✅ recibe el dato como parámetro
```

🚫 El método sin parámetros no es reutilizable — depende del contexto externo



¡Cerramos la Unidad I!

- ✓ Variables, constantes y tipos de datos
- ✓ Operadores y estructuras de control
- ✓ Arreglos, matrices y List<T>
- ✓ Métodos: DRY, parámetros, retorno, sobrecarga

Resumen — Semana 4

- ✓ **DRY** — Don't Repeat Yourself — código repetido = método pendiente.
- ✓ **void** — El método actúa (imprime, guarda) pero no devuelve resultado.
- ✓ **Tipo retorno** — double, int, bool... — el método calcula y devuelve con return.
- ✓ **Parámetros** — Datos que el método recibe. La llamada pasa argumentos.
- ✓ **Sobrecarga** — Mismo nombre, distinta firma. C# elige según los argumentos.
- ✓ **Valor por defecto** — `static void M(int n = 5)` — si no se pasa, usa 5.
- ✓ **Descomposición** — Un método = una responsabilidad. Nombre descriptivo.

Para la próxima clase — Guía de ejercicios + Plataforma

E1

Fácil

Funciones básicas de conversión

E2

Fácil

Método de validación de rango

E3

Fácil

Métodos de formato de texto

E4

Medio

Calculadora de geometría

E5

Medio

Refactorizar programa de arreglos

E6

Medio

Método de ordenamiento (bubble sort)

E7

Difícil

Mini sistema de calificaciones

E8

Difícil

Calculadora científica modular



CodeQuest C# — Lección 4: Métodos [Pega aquí el enlace a la plataforma]